



Hyperparameter Tuning

A practical guide to hyperparameter optimization for regression models using the `daltoolbox` package in R. The `reg_tune` function performs a grid search over the parameter space, maximizing predictive performance and preventing overfitting — all within the same stable experimental workflow introduced in the previous sections.

MODEL SELECTION

HYPERPARAMETER TUNING

REG_TUNE / DALTOOLBOX

Hyperparameter Tuning & Grid Search

Hyperparameters are configuration values that **cannot be learned from data** — they must be defined *before* training begins. Unlike model parameters (e.g., regression coefficients), hyperparameters control the learning process itself, such as kernel type, regularization strength, or tree depth. Choosing them well is critical to model generalization.

Parameters vs. Hyperparameters

Parameters are learned during `fit()` from training data (e.g., weights, coefficients). **Hyperparameters** are set before `fit()` and govern how the model learns — they are not updated by the optimization algorithm.

Grid Search

Grid Search exhaustively evaluates **all combinations** of a predefined hyperparameter space. Each candidate configuration is trained and scored against a performance criterion (e.g., MSE, R^2). The configuration yielding the best score is selected as the optimal model.

`reg_tune` in `daltoolbox`

`reg_tune` wraps any regression model and performs this search **transparently**. It accepts a model object and a `ranges` list defining the search space. Crucially, it integrates into the existing Experiment Line structure — `fit()` and `predict()` calls remain unchanged.

Why It Matters

Poor hyperparameter choices lead to **underfitting** (too simple) or **overfitting** (too complex). Systematic tuning ensures the model configuration is chosen based on evidence from the data, not manual trial-and-error, improving both performance and reproducibility.

 Ref: Bergstra, J. & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13, 281–305.

reg_tune: How It Works

The `reg_tune` function streamlines hyperparameter optimization within the `daltoolbox` framework. It systematically explores a predefined search space to identify the best model configuration, ensuring seamless integration into the standard experiment line.

01

Define Search Space

The `ranges` argument, a named list, outlines specific values or ranges for each hyperparameter (e.g., cost, kernel type). This forms a Cartesian product of all candidate configurations.

03

Select Optimal Configuration

Based on the defined performance criterion (e.g., minimizing RMSE or maximizing R^2), the function identifies and stores the hyperparameter combination that yielded the best results.

02

Iterative Model Evaluation

For every combination generated from the search space, `reg_tune` internally calls the `fit()` method on the provided regression model, then evaluates its predictive performance on the training data.

04

Return Tuned Model

After the comprehensive search, the `fit(tune, train_data)` call returns a model object already configured with these optimal hyperparameters, making it immediately ready for prediction with the best settings found.

③ Ref: Ogasawara et al. (2026). `daltoolbox`: Leveraging Experiment Lines to Data Analytics. R package version 1.3.767. CRAN.

reg_tune: Hyperparameter Search for Regression

Setup & Data

```
# Regression Tuning

library(daltoolbox)
library(MASS)

data(Boston)
Boston <- as.matrix(Boston)

set.seed(1)
sr <- sample_random()
sr <- train_test(sr, Boston)
boston_train <- sr$train
boston_test <- sr$test
```

Tuning & Evaluation

```
# Hyperparameter grid + search
tune <- reg_tune(reg_svm("medv"),
  ranges = list(
    seq(0,1,0.2),
    cost=seq(20,100,20),
    kernel=c("radial"))
model <- fit(tune, boston_train)

# Training evaluation (best params)
train_pred <- predict(model, boston_train)
train_eval <- evaluate(model,
  boston_train[, "medv"], train_pred)
print(train_eval$metrics)
# mse=2.39, smape=0.05, R2=0.97

# Test evaluation
test_pred <- predict(model, boston_test)
test_eval <- evaluate(model,
  boston_test[, "medv"], test_pred)
print(test_eval$metrics)
# mse=13.61, smape=0.13, R2=0.77
```

Step-by-Step

- Wrap the base model in `reg_tune()` and define the search grid with `ranges`
- The grid explores combinations of `epsilon`, `cost`, and `kernel`
- `fit(tune, boston_train)` trains and selects the best hyperparameters
- Result: test $R^2=0.77$ — an improvement over manual SVM ($R^2=0.76$)
- The same `predict` → `evaluate` pattern is preserved without modification

❏ **Key Concept:** `reg_tune` encapsulates grid search without changing the experimental logic. The **Experiment Line remains stable**. Ref: Kohavi (1995).

Key Hyperparameters by Model

Each regression model has specific hyperparameters that critically influence its performance and generalization. Understanding their roles is key to effective tuning.

SVM (reg_svm)

Epsilon: Width of the insensitive tube, controlling tolerance to errors.

Cost (C): Regularization parameter, balancing model complexity and training error (high C = low bias, high variance).

Kernel: (Linear, Radial, Polynomial, Sigmoid)
Defines the feature space transformation.

KNN (reg_knn)

K: The number of nearest neighbors. Low k results in high variance and low bias (overfitting); high k leads to high bias and low variance (underfitting).

MLP (reg_mlp)

Size: Number of hidden units, influencing model capacity. More units generally mean lower bias and higher variance.

Decay: L2 regularization strength, penalizing large weights to prevent overfitting (increases bias, reduces variance).

Random Forest (reg_rf)

Mtry: Number of features sampled at each split, controlling tree diversity (lower mtry reduces variance).

Ntree: Number of decision trees in the forest. More trees increase stability and reduce variance, but with diminishing returns.

📌 **Critical Insight:** Choosing the right search range for each hyperparameter is as important as the search itself — too narrow a range may miss the optimum; too wide increases computational cost.

Hyperparameter Grids for Other Models

Grids: SVM and KNN

```
# SVM — epsilon, cost, kernel
ranges <- list(
  seq(0,1,0.2),
  cost=seq(20,100,20),
  kernel=c("linear","radial",
           "polynomial","sigmoid"))

# KNN — number of neighbors
ranges <- list(k=1:20)
```

Grids: MLP and Random Forest

```
# MLP — hidden units + regularization
ranges <- list(
  size=1:10,
  decay=seq(0, 1, 0.1))

# Random Forest — mtry + ntree
ranges <- list(
  mtry=1:10,
  ntree=1:10)
```

Step-by-step

- Replace the base model in `reg_tune()` with any desired model
- **SVM:** vary `epsilon`, `cost`, and `kernel`
- **KNN:** search for the best `k` in the range 1–20
- **MLP:** combine `size` (1–10) with `decay` (0–1)
- **RF:** explore `mtry` and `ntree` simultaneously

❏ **Key concept:** The same `reg_tune` utility works for any regression model in `daltoolbox`. Only the **base model and the grid change** — the experimental workflow remains identical, ensuring a consistent and reproducible modeling pipeline.

Common Mistakes in Regression

Confusing regression metrics with accuracy

MSE, RMSE, and MAE are not percentage scores. Interpreting them with a classification mindset leads to incorrect conclusions.

Ignoring the scale of the target variable

An RMSE of 5 can be excellent or terrible depending on the scale of medv. Always contextualize the error relative to the target variable's range.

Treating low training error as good prediction

$R^2=0.98$ on training data (Random Forest) does not guarantee generalization. Always evaluate performance on the test set.

What to watch for in the workflow

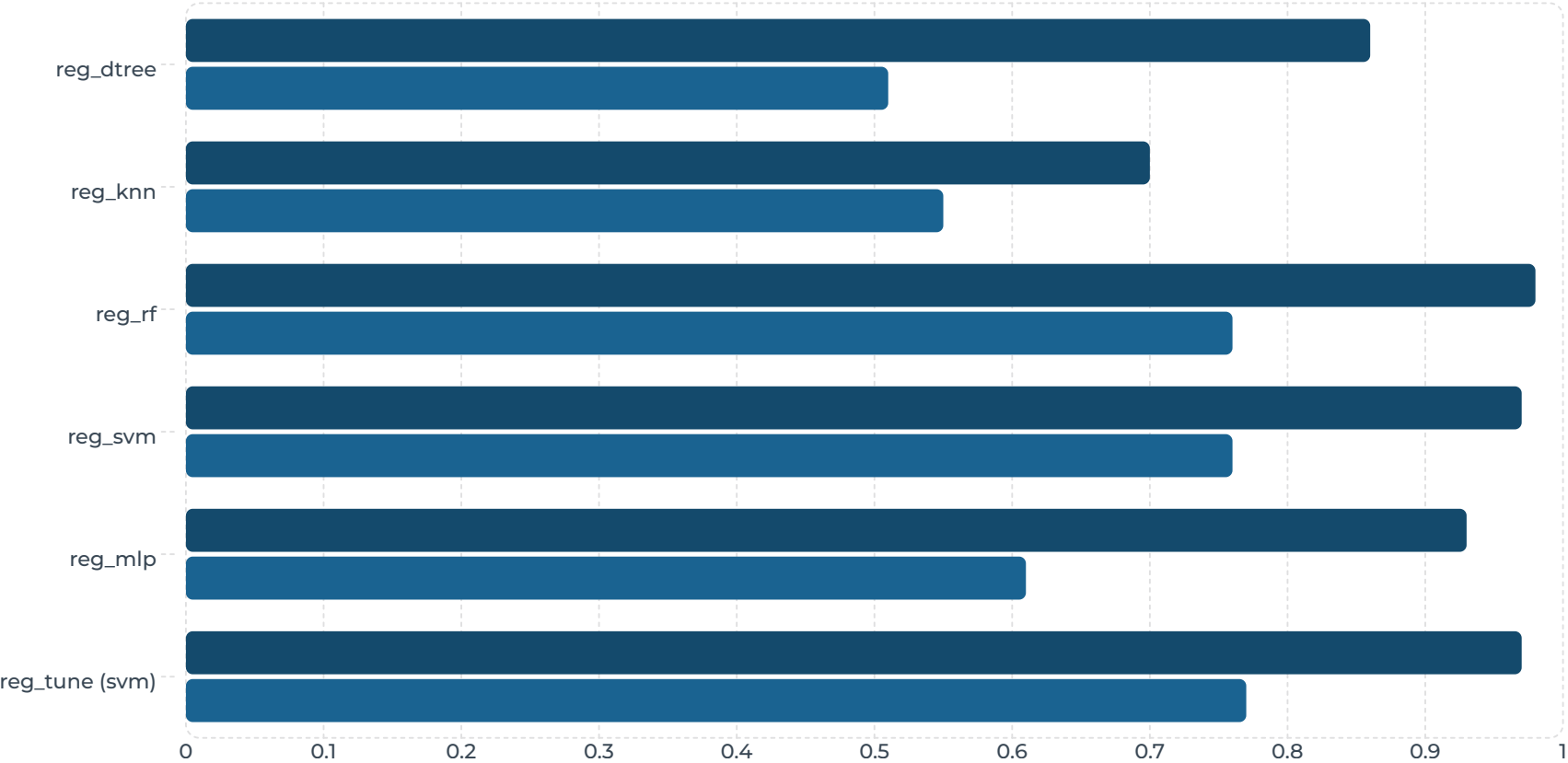
- The learner definition remains compact even with a numeric target variable
- The key conceptual shift lies in **evaluation**, not in fitting
- Always compare train vs. test performance to detect overfitting
- The Experiment Line stays consistent across all examples

📌 **Key concept:** In regression, the goal is to minimize **prediction error on the test set**, not to maximize fit on training data. Core metrics: **MSE** (sensitive to outliers), **SMAPE** (relative error), and **R^2** (explained variance).

Performance Comparison: All Models

■ R² Treino ■ R² Teste

Modelo



With the common pitfalls in mind, this comparison shows how each model truly performs on unseen data. Random Forest and SVM (both manual and tuned) lead in generalization (R² test $\approx$ 0.76–0.77). The Decision Tree shows the largest train/test gap, confirming overfitting. Hyperparameter tuning yields a marginal improvement for SVM. Crucially, the **Experiment Line remains identical** across all models — only the learner and evaluation metrics change.

Summary

This presentation focused on **hyperparameter tuning for regression models** using `reg_tune` in `daltoolbox`. The key insight: tuning integrates seamlessly into the standard experimental flow — grid search over a Cartesian product of ranges selects the best configuration without altering the pipeline structure.

Stable Experiment Line

`reg_tune` integrates directly into the standard flow: load → split → fit → predict → evaluate. The pipeline structure remains identical — only the learner and its hyperparameter grid change.

Grid Search is Systematic

Internally, `reg_tune` performs an exhaustive search over the **Cartesian product** of all specified ranges, evaluating every combination to select the configuration that minimizes error on validation data.

Tuning Improved SVR

Tuning `reg_svm` over epsilon, cost, and kernel on the Boston dataset raised test R^2 from **0.756 → 0.774** — the best result across all six models compared in this presentation.

Model-Specific Grids

Each model has its own key hyperparameters to tune: **SVM** (epsilon, cost, kernel), **KNN** (k), **MLP** (size, decay), **Random Forest** (mtry, ntree). Choosing the right grid is critical for meaningful gains.

References & Additional Sources

Academic references and online resources covering hyperparameter tuning, regression models, and the daltoolbox framework.

Online Resources



CRAN Package – daltoolbox

cran.r-project.org/web/packages/daltoolbox

DAL Toolbox – GitHub

github.com/cefet-rj-dal/daltoolbox

GitHub Repository

github.com/cefet-rj-dal/daltoolbox

Source code, issues, and contributions.

Key Academic References

- Ogasawara, E., Paulino, C., Murta, L., Werner, C. & Mattoso, M. (2009). Experiment Line: Software Reuse in Scientific Workflows. *SSDBM 2009, LNCS 5566*, pp. 264–272. Springer.
- Bergstra, J. & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13, 281–305.
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *IJCAI*, 14(2), 1137–1145.
- Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning. 2nd ed. Springer.